

Python for Data Science, Statistics, and Data Visualization

CIS 635 — Knowledge Discovery & Data Mining

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [10] this material when using or reproducing it.

Unit: Foundations of KDD and Python Programming

Contents

1	Learning Objectives	3
2	From the KDD Process to a Data Science Toolkit	3
3	NumPy for Numerical Computing	4
3.1	Creating and Inspecting Arrays	4
3.2	Vectorized Operations and Why They Are Faster	5
3.3	Indexing, Slicing, and Boolean Masking	5
3.4	Aggregations and Broadcasting	6
4	Pandas for Data Loading, Manipulation, and Analysis	7
4.1	Series and DataFrames	7
4.2	Loading Data	8
4.3	Selecting, Filtering, and Sorting	8
4.4	Handling Missing Data and Duplicates	9
4.5	Grouping and Aggregating	9
5	Descriptive Statistics	10
5.1	Measures of Central Tendency	10
5.2	Measures of Variability	11
5.3	A Simple, Robust Outlier Rule	11
5.4	Comparing Across Groups	12
6	Exploratory Data Analysis (EDA)	13
6.1	Profiling a Freshly Loaded Dataset	13
6.2	Systematic Outlier Detection Across Every Group	14
6.3	From EDA Back to Interpretation	14

7	Principles of Data Visualization	15
7.1	Choosing the Right Chart Type	15
7.2	A Concrete Illustration of a Common Pitfall	16
7.3	Best Practices and Common Pitfalls	16
8	Matplotlib for Static Visualization	17
8.1	Basic Plotting	17
8.2	Bar Charts and the Mean-Median Gap, Made Visible	18
8.3	Histograms: Seeing a Distribution's Shape	19
8.4	Scatter Plots and Subplots: A Real Dataset	19
9	Plotly for Interactive Visualization	21
9.1	A First Interactive Chart	21
9.2	Interactive Bar and Line Charts	22
9.3	Choosing Between Matplotlib and Plotly	24
10	Best Practices and Common Pitfalls	24
11	A Practical Mental Model	25
12	Capstone Exercise: An End-to-End EDA and Visualization Pipeline	25
13	Summary	27
14	Discussion Questions	28
15	Key Takeaways for CIS 635	28

1 Learning Objectives

After completing this reading, you should be able to:

- Explain how NumPy, Pandas, descriptive statistics, exploratory data analysis, and data visualization each support a specific stage of the KDD process introduced in the previous reading.
- Create, index, slice, and perform vectorized operations on NumPy arrays, and explain why vectorization is faster than an equivalent Python loop.
- Load, inspect, clean, filter, sort, group, and aggregate tabular data using Pandas Series and DataFrames.
- Compute and interpret measures of central tendency (mean, median, mode) and variability (range, variance, standard deviation, interquartile range), and explain when each is more informative than the others.
- Carry out a systematic exploratory data analysis (EDA) pass on a new dataset, including profiling its structure, detecting missing values and duplicates, and flagging candidate outliers.
- Identify common data visualization pitfalls (misleading axes, poor chart choice, uninformative color) and apply basic principles for honest, readable charts.
- Create static visualizations (line, bar, histogram, scatter, box plot) with Matplotlib, and interactive visualizations with Plotly, and choose appropriately between them.
- Carry out a complete, small-scale exploratory data analysis and visualization pipeline on a real dataset, connecting each step back to the KDD process.

2 From the KDD Process to a Data Science Toolkit

The previous reading introduced the KDD process — Selection, Preprocessing, Transformation, Data Mining, and Interpretation and Evaluation —[1] and reviewed the core Python language needed to implement it. Every example in that reading used only plain Python: lists, dictionaries, loops, and functions. That was a deliberate choice, to make the *process* visible without a library doing work behind the scenes. In practice, however, almost nobody performs Selection, Preprocessing, and Transformation on a real dataset using bare Python loops — the data is too large, and the code becomes too verbose. This reading introduces the tools practicing data scientists actually use for that work.

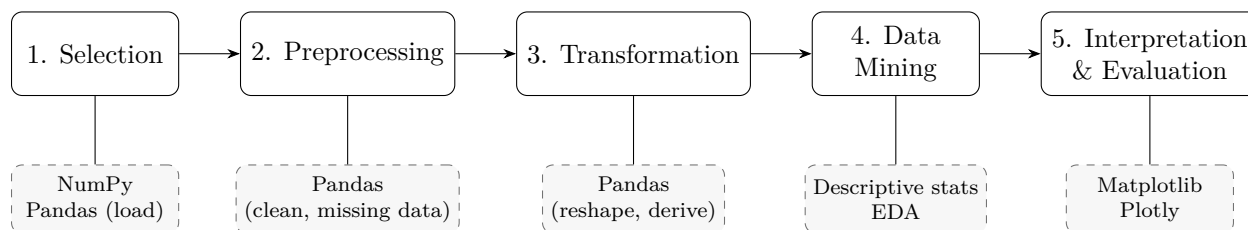


Figure 1: Where this reading’s tools fit within the KDD process introduced in the previous reading.

NumPy and Pandas do most of the work of Selection, Preprocessing, and Transformation; descriptive statistics and exploratory data analysis (EDA) support Data Mining by revealing

patterns worth pursuing; and Matplotlib and Plotly support Interpretation and Evaluation by making those patterns visible to a human audience.

This mapping is a guide, not a strict rule — Pandas is also used during EDA, and a visualization can reveal a data-quality problem that sends you back to Preprocessing, just as the previous reading’s feedback loops would predict. Keep Figure 1 in mind as an organizing picture as this reading moves through each tool in turn.

Key Idea

Every dataset and finding in this reading traces back to code you could run yourself, and several examples reuse the same small three-store retail dataset throughout, so that a data-quality issue introduced early (Section 4) keeps reappearing — in the descriptive statistics (Section 5), the EDA pass (Section 6), and the visualizations (Section 8) — exactly as it would in a real analysis, rather than disappearing conveniently between sections.

Practical Exercises

1. Without looking ahead, predict which KDD stage(s) you think Matplotlib and Plotly primarily support, and why. Then check your prediction against Figure 1.
2. The previous reading’s farm-sensor and bookstore examples used only plain Python lists and dictionaries. Name one specific limitation you noticed (or can imagine) that would make this approach impractical for a dataset with 50,000 rows instead of 8 or 10.
3. Figure 1 places “descriptive stats” and “EDA” under Data Mining, but the text notes this mapping is not strict. Describe one realistic scenario in which an EDA finding would send an analyst backward to Preprocessing instead of forward to Interpretation.

3 NumPy for Numerical Computing

NumPy (Numerical Python) provides the `ndarray`, a fixed-type, grid-shaped array that supports fast, **vectorized** operations[2] — applying a computation to every element at once, without an explicit Python loop. Pandas, covered in Section 4, is built directly on top of NumPy arrays[3], which is why understanding NumPy first makes Pandas far less mysterious.

3.1 Creating and Inspecting Arrays

This section’s running example is a week of hourly outdoor-temperature readings from a single environmental-monitoring station: 7 days times 24 hours is naturally a two-dimensional grid, which is exactly what a NumPy array is built to represent.

```
import numpy as np

rng = np.random.default_rng(42)
hours = np.arange(24)
daily_pattern = 18 + 6 * np.sin((hours - 9) * np.pi / 12) # warmest mid-afternoon
week_readings = np.array([daily_pattern + rng.normal(0, 1.0, size=24) for _ in range(7)])
```

```
week_readings.shape = (7, 24)
week_readings.dtype = float64
week_readings[0, :5] (Monday, first 5 hours) = [14.06 11.76 12.95 12.94 10.25]
```

Every NumPy array has a **shape** (its dimensions) and a **dtype** (the single data type shared by every element — unlike a Python list, which can freely mix types). A handful of constructors cover most everyday needs:

```
zeros = np.zeros(5)
ones_2d = np.ones((2, 3))
evenly_spaced = np.linspace(0, 100, 5)
```

```
np.zeros(5) = [0. 0. 0. 0. 0.]
np.ones((2, 3)) =
[[1. 1. 1.]
 [1. 1. 1.]]
np.linspace(0, 100, 5) = [ 0. 25. 50. 75. 100.]
```

3.2 Vectorized Operations and Why They Are Faster

Converting every temperature reading from Celsius to Fahrenheit can be written as a nested loop, exactly as Document 1 would have — or as a single vectorized expression:

```
def celsius_to_fahrenheit_loop(values):
    result = []
    for row in values:
        result.append([v * 9 / 5 + 32 for v in row])
    return result

week_fahrenheit_vectorized = week_readings * 9 / 5 + 32
week_fahrenheit_loop = celsius_to_fahrenheit_loop(week_readings.tolist())
```

```
vectorized[0, :3] = [57.31 53.17 55.32]
loop[0][:3]       = [57.31, 53.17, 55.32]
```

Both approaches produce identical results, but not at the same speed:

```
Timing on 1,000,000 values:
  vectorized (NumPy): 5.8 ms
  loop (pure Python): 369.5 ms
  speedup: 64x
```

Key Idea

The speedup exists because a vectorized NumPy operation runs a single, pre-compiled loop written in C across contiguous memory, while a Python **for** loop pays Python’s per-iteration interpreter overhead 1,000,000 separate times. The exact multiplier will vary by machine, but the qualitative result — vectorized code substantially outperforming an equivalent Python loop on large arrays — is one of the most consistent findings in scientific Python, and it is the entire reason Pandas, and most of the scientific Python ecosystem, is built on NumPy rather than plain lists.

3.3 Indexing, Slicing, and Boolean Masking

A two-dimensional array is indexed `array[row, column]`, and slices work exactly as they do for lists (Document 1, Section 9.1), just along each dimension independently.

```

week_readings[2, 14]    # Wednesday, 2pm
week_readings[:, 0]     # all days, midnight (column 0)
week_readings[3]        # all of Thursday (row 3)

```

```

week_readings[2, 14] (Wednesday, 2pm) = 23.96
week_readings[:, 0] (all days, midnight) = [14.06 13.33 14.44 12.84 12.43 12.73 12.94]
week_readings[3] (all of Thursday) shape = (24,)

```

Boolean masking selects elements matching a condition, without writing a filtering loop:

```

hot_mask = week_readings > 20
print(hot_mask.sum())           # count of readings above 20C
print(week_readings[hot_mask])  # the actual matching values

```

```

number of hourly readings above 20C: 66
actual values above 20C: [20.43 21.78 22.31 24.32 24.26 23.14 24.16 22.24] ... (first 8
    shown)

```

Tip

`array[condition]` is the NumPy equivalent of the list-comprehension filter from Document 1, Section 7.4 (`[x for x in values if condition]`) — same idea, vectorized. You will use this pattern constantly once Pandas is introduced in Section 4, since a Pandas DataFrame's filtering syntax is built directly on top of it.

3.4 Aggregations and Broadcasting

NumPy's aggregation methods (`.mean()`, `.min()`, `.max()`, `.sum()`, and others) accept an `axis` argument specifying which dimension to collapse.

```

week_readings.mean()           # overall mean, one number
week_readings.mean(axis=1)      # mean per day (collapse the hour axis)
week_readings.mean(axis=0)      # mean per hour-of-day, across all days

```

```

overall mean temperature: 17.95
mean temperature per day (axis=1): [17.98 18.18 17.91 17.84 17.79 18.02 17.91]
mean temperature per hour-of-day, across all 7 days (axis=0):
    [13.25 12.51 12.45 12.42 11.93 12.92] ... (first 6 hours shown)
min: 10.25, max: 24.65

```

Broadcasting lets NumPy combine arrays of different (but compatible) shapes without writing a loop. Here, a single per-day calibration offset (shape `(7,)`) is applied to every hourly reading in that day (shape `(7, 24)`):

```

calibration_offset_per_day = np.array([0.5, -0.2, 0.0, 0.3, -0.1, 0.4, 0.1])
calibrated = week_readings + calibration_offset_per_day[:, np.newaxis]

```

```

calibration_offset_per_day.shape = (7,)
week_readings.shape = (7, 24)
calibrated[0, :3] (Monday, offset +0.5) = [14.56 12.26 13.45]
original[0, :3]                        = [14.06 11.76 12.95]

```

Common Pitfall

`calibration_offset_per_day[:, np.newaxis]` reshapes the 1-D array of shape (7,) into a 2-D column of shape (7, 1), which is what allows NumPy to broadcast it against each row of the (7, 24) array. Forgetting `np.newaxis` here (or using the wrong axis) is a common source of silent shape-mismatch bugs — always check `.shape` after a broadcasting operation you are not fully sure about.

Practical Exercises

1. Create a NumPy array of 30 daily rainfall totals (in millimeters) for a single month, using `rng.exponential(5, size=30)` with a seeded generator. Compute its mean, its maximum, and the number of days with more than 10mm of rain, using vectorized operations (no loop).
2. Reshape the 30-day rainfall array from the previous exercise into a (5, 6) array representing 5 weeks of 6 days each (drop or pad as needed), then compute the total rainfall per week using `.sum(axis=...)` with the correct axis.
3. Using `week_readings` from this section, write a boolean mask that identifies hours where the temperature was *simultaneously* above 20°C *and* it was one of the first 12 hours of the day (hint: you will need to combine a value-based condition with a position-based condition using `&`, being careful about shapes).
4. Time a vectorized computation of $x^2 + 3x - 5$ over an array of 2,000,000 random values against an equivalent pure-Python loop over a plain list of the same values, using Python's `time.perf_counter()` as this section did. Report the speedup you observe, and note that it may differ from this section's number — explain in one sentence why that is expected.
5. A colleague suggests broadcasting a shape-(24,) array of “expected hourly baseline temperatures” against `week_readings` (shape (7, 24)) to compute each hour's deviation from baseline, *without* reshaping the baseline array first. Will this work directly, or does it need reshaping like the calibration example did? Explain using NumPy's broadcasting rule (dimensions are compared from the right, and must either match or be 1).

4 Pandas for Data Loading, Manipulation, and Analysis

Pandas is the standard Python library for working with tabular data[3]: rows and named columns, mixed data types, labeled indices, and built-in support for exactly the operations the KDD process needs — loading, selecting, cleaning, transforming, and aggregating. This section's running example is a synthetic but realistic dataset: 30 days of sales at each of three stores in a small regional retail chain, deliberately seeded with the same kinds of data-quality problems — missing values, a duplicate record, and an outlier — that Document 1's farm-sensor example introduced with plain Python.

4.1 Series and DataFrames

A **Series** is a single labeled column of data; a **DataFrame** is a table of Series sharing a common index.

```
import pandas as pd

daily_sales_series = pd.Series([412.50, 389.00, 455.75, 402.10],
```

```
index=["Mon", "Tue", "Wed", "Thu"], name="sales")
```

```
Mon    412.50
Tue    389.00
Wed    455.75
Thu    402.10
Name: sales, dtype: float64

daily_sales_series['Wed'] = 455.75
```

4.2 Loading Data

Pandas reads directly from a CSV file into a DataFrame with a single call:

```
df = pd.read_csv("regional_store_sales.csv", parse_dates=["date"])
```

```
Loaded DataFrame: 91 rows, 4 columns
   date      store  daily_sales  num_transactions
0 2026-06-01  Downtown      1800.18              73
1 2026-06-02  Downtown      1760.52              65
2 2026-06-03  Downtown      1734.53              64
3 2026-06-04  Downtown      1808.66              79
4 2026-06-05  Downtown      1729.12              66
```

Per this course's decision to draw on both synthetic and real, bundled datasets, it is worth seeing how easily a real dataset becomes a DataFrame too. Several well-known small datasets ship directly with scikit-learn, useful for practice and for comparing techniques against a widely used benchmark:

```
from sklearn.datasets import load_wine
wine_bunch = load_wine()
wine_df = pd.DataFrame(wine_bunch.data, columns=wine_bunch.feature_names)
wine_df["cultivar"] = pd.Categorical.from_codes(wine_bunch.target, wine_bunch.
    target_names)
```

```
wine_df.shape = (178, 14)
   alcohol  malic_acid  color_intensity  cultivar
0    14.23      1.71          5.64  class_0
1    13.20      1.78          4.38  class_0
2    13.16      2.36          5.68  class_0
3    14.37      1.95          7.80  class_0
4    13.24      2.59          4.32  class_0
```

This dataset — 178 wine samples from three cultivars, each with 13 chemical measurements —^[9] returns later in this reading's capstone (Section 12) and appears again in the Matplotlib and Plotly sections.

4.3 Selecting, Filtering, and Sorting

```
df[["date", "store", "daily_sales"]].head(3)

weekend_riverside = df[(df["store"] == "Riverside") & (df["daily_sales"] > 1200)]
```



```
top5 = df.sort_values("daily_sales", ascending=False).head(5)
```

```
Riverside days with sales > 1200: 19 rows
```

```
Top 5 sales days:
```

	date	store	daily_sales
65	2026-06-06	Hilltop	11514.60
5	2026-06-06	Downtown	2431.70
12	2026-06-13	Downtown	2369.34
20	2026-06-21	Downtown	2360.68
6	2026-06-07	Downtown	2359.73

Common Pitfall

Look closely at that Hilltop value: \$11,514.60 on a single day, roughly 10 times every other value in the table. This is not a typo in this reading — it is a deliberately injected data-entry error (an extra digit), included so that Sections 5, 6, and 8 can each show what a real analysis does when it encounters exactly this kind of problem, rather than working with data that has been quietly cleaned before you ever saw it.

4.4 Handling Missing Data and Duplicates

```
df["daily_sales"].isna().sum()
df.duplicated().sum()

df_no_dupes = df.drop_duplicates()
df_clean = df_no_dupes.dropna(subset=["daily_sales"])
```

```
missing daily_sales values: 4
fully duplicated rows: 1
shape after drop_duplicates(): (90, 4)
shape after also dropping missing daily_sales: (86, 4)
```

Tip

`dropna()` is the simplest way to handle missing values, but it is not always the right one — it discards the entire row, including any other, perfectly good measurements it contains. Document 1's Preprocessing discussion (Section 4) and this reading's EDA section (Section 6) both return to this trade-off: dropping, imputing (filling in an estimate), and flagging missing data are three different policies with three different risks, and the right choice depends on *why* the value is missing.

4.5 Grouping and Aggregating

`groupby()` is Pandas' equivalent of the dictionary-accumulator pattern from Document 1's capstone (Section 12) — but vectorized, and with many aggregation functions built in.

```
store_summary = df_clean.groupby("store")["daily_sales"].agg(["mean", "median", "std", "count"])
```

	mean	median	std	count
store				
Downtown	1897.71	1797.98	280.60	30
Hilltop	1394.22	956.79	2028.54	27
Riverside	1289.88	1224.94	158.08	29

Notice Hilltop's numbers already hint at a problem worth investigating: its mean (\$1,394.22) is noticeably higher than its median (\$956.79), and its standard deviation (\$2,028.54) dwarfs the other two stores'. Section 5 explains exactly why that gap is a diagnostic signal, not a coincidence.

Practical Exercises

1. Load `regional_store_sales.csv` yourself (or reconstruct a similar dataset), and write code to find the single day with the *lowest* `daily_sales` value at each store, using `groupby()` combined with an appropriate aggregation or filtering approach.
2. The Downtown store's data has no missing values or duplicates. Write a filtering expression that selects only Downtown's rows where `num_transactions` is above the Downtown-only mean number of transactions (you will need to compute that mean first).
3. Using scikit-learn's `load_wine()` dataset, load it into a `DataFrame` as shown in this section, then write code to select only the rows belonging to "class_2" and sort them by "proline" in descending order.
4. Compare `dropna()` versus filling missing `daily_sales` values with the *median* sales for that store (hint: `groupby("store")["daily_sales"].transform("median")`, combined with `.fillna()`). Recompute the store-level means both ways, and discuss which policy you would recommend to a store manager and why.
5. Using `df.duplicated(keep=False)`, find and display *both* copies of the duplicated row in the retail dataset (not just a count). Explain, referencing the KDD process, which stage is responsible for catching a data problem like this one before it reaches Data Mining.

5 Descriptive Statistics

Descriptive statistics summarize a dataset with a small number of representative values. They are the simplest form of Data Mining in the KDD process — and, as this section shows concretely, the first line of defense against being misled by a single bad data point.

5.1 Measures of Central Tendency

The **mean** (arithmetic average), **median** (middle value when sorted), and **mode** (most frequent value) all describe “a typical value,” but can disagree sharply when a distribution is skewed or contains outliers.

```
hilltop = df[df["store"] == "Hilltop"]["daily_sales"].dropna()
hilltop.mean()
hilltop.median()
df[df["store"]=="Hilltop"]["num_transactions"].mode()
```

```
mean:    1394.22
median:  956.79
mode of num_transactions (Hilltop): [36]
```

Common Pitfall

The mean and median disagree by more than \$400 for Hilltop. This is exactly the pattern flagged at the end of Section 4: the mean is **sensitive to outliers** (a single \$11,514.60 value pulls the average up substantially), while the median — the middle value when all 27 readings are sorted — is not. When you see a mean and median that disagree sharply, that disagreement is itself a finding worth investigating before reporting either number on its own.

5.2 Measures of Variability

The **range** (max – min), **variance** (average squared deviation from the mean), **standard deviation** (the square root of variance, in the original units), and **interquartile range (IQR)** (the range of the middle 50% of the data, $Q_3 - Q_1$) all describe how spread out a distribution is.

```
hilltop.max() - hilltop.min()      # range
hilltop.var()                     # sample variance
hilltop.std()                     # sample standard deviation

q1, q3 = hilltop.quantile(0.25), hilltop.quantile(0.75)
iqr = q3 - q1
```

```
range: 10726.37
variance (sample): 4114964.25
standard deviation (sample): 2028.54
Q1 (25th percentile): 902.33
Q3 (75th percentile): 1091.22
IQR (Q3 - Q1): 188.90
```

The standard deviation (\$2,028.54) is larger than the median itself (\$956.79) — a strong signal that something unusual is happening in this data. Removing just the one confirmed data-entry error changes the picture dramatically:

```
With the outlier included:    std = 2028.54
With the outlier removed:    std = 158.46
(n went from 27 to 26)
```

Key Idea

The IQR, unlike variance and standard deviation, is a **robust** statistic: because it is based on the middle 50% of sorted values, a single extreme outlier barely moves it. This robustness is exactly why the IQR is used as the basis for a standard outlier-detection rule, applied next.

5.3 A Simple, Robust Outlier Rule

A common convention flags any value more than $1.5 \times \text{IQR}$ below Q_1 or above Q_3 as a candidate outlier:

```
lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr
flagged = hilltop[(hilltop < lower_fence) | (hilltop > upper_fence)]
```

```
IQR outlier fences: [618.98, 1374.57]
Values flagged as outliers: [11514.6, 1398.42]
```

Common Pitfall

The rule flags *two* values, not one. The \$11,514.60 entry is almost certainly the injected data-entry error. But \$1,398.42 is also flagged, and there is no code-based way to know whether that is a second error or simply a genuinely strong sales day. This is precisely the limit of an automated rule: it narrows a list of 27 values down to 2 candidates worth a human's attention, but confirming which of those candidates is actually a problem is Interpretation and Evaluation's job, not the rule's.

5.4 Comparing Across Groups

	mean	median	std
store			
Downtown	1897.71	1797.98	280.60
Hilltop	1394.22	956.79	2028.54
Riverside	1289.88	1224.94	158.08

Downtown and Riverside both show mean and median within a few percent of each other, and standard deviations proportional to their typical sales level — the pattern you would expect from a distribution without major outliers. Hilltop is the visible exception, illustrating a genuinely useful habit: scanning mean-versus-median gaps across every group in a dataset, rather than only checking the one group you happen to be looking at.

Practical Exercises

1. Compute the mean, median, variance, standard deviation, and IQR for the Downtown store's `daily_sales` (which has no known outlier), and compare the relative size of Downtown's standard deviation to Hilltop's. Express Hilltop's std as a multiple of Downtown's, and discuss what that multiple tells you.
2. Using the `num_transactions` column instead of `daily_sales`, compute the mode for all three stores. Is there more than one mode for any store (a "multimodal" result)? If Pandas' `.mode()` returns more than one value, explain what that means about the underlying data.
3. Recompute the IQR-based outlier fences for Hilltop *after* removing the confirmed \$11,514.60 error, and check whether \$1,398.42 is still flagged. What does this tell you about how sensitive the IQR fences themselves are to the presence of a known outlier?
4. Using scikit-learn's wine dataset (Section 4), compute the mean, median, and standard deviation of the "proline" feature for each of the three cultivars. Is there a cultivar where mean and median disagree noticeably, the way Hilltop's did? If so, investigate further (a histogram, covered in Section 8, may help) and describe what you find.
5. A classmate argues that since the mean is "pulled around" by outliers, analysts should always prefer the median and ignore the mean entirely. Write a short response (4–6 sentences) either agreeing, disagreeing, or partially agreeing, with a concrete scenario supporting your position.

6 Exploratory Data Analysis (EDA)

Exploratory data analysis (EDA) is the systematic practice of examining a dataset's structure, quality, and patterns[4] *before* committing to a specific analysis or model — combining Selection, Preprocessing, and the early part of Data Mining into a single, repeatable habit. Sections 4 and 5 already performed pieces of EDA on the retail dataset without naming it; this section makes the workflow explicit and systematic.

6.1 Profiling a Freshly Loaded Dataset

Three Pandas methods answer the questions any EDA pass should start with: *what columns exist, what type is each one, and are there any missing values?*

```
df = pd.read_csv("regional_store_sales.csv", parse_dates=["date"])
df.info()
df.describe()
df.isna().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 91 entries, 0 to 90
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   date             91 non-null    datetime64[us]
1   store            91 non-null    str
2   daily_sales      87 non-null    float64
3   num_transactions 91 non-null    int64
dtypes: datetime64[us](1), float64(1), int64(1), str(1)
memory usage: 3.0 KB

Missing values per column:
date            0
store           0
daily_sales     4
num_transactions 0
```

`df.info()` immediately tells you three things worth checking before anything else: the `daily_sales` column has only 87 of 91 non-null values (4 are missing), every column has the type you would expect, and there are 91 rows total. `df.describe()` adds distributional context:

	date	daily_sales	num_transactions
count	91	87.00	91.00
mean	2026-06-15 10:48:47.472527	1534.67	54.64
min	2026-06-01 00:00:00	788.23	27.00
25%	2026-06-08 00:00:00	1098.11	40.50
50%	2026-06-15 00:00:00	1311.95	52.00
75%	2026-06-23 00:00:00	1731.82	65.00
max	2026-06-30 00:00:00	11514.60	99.00
std	NaN	1162.38	17.41

Key Idea

Look at `daily_sales`' 75th percentile (\$1,731.82) versus its maximum (\$11,514.60): a jump of nearly 10x from the 75th percentile to the maximum, with nothing between the 75th percentile and the top value shown here, is a strong, immediate red flag from `describe()` alone — often visible before you have written a single line of outlier-detection code.

6.2 Systematic Outlier Detection Across Every Group

Section 5 applied the IQR rule to Hilltop specifically, because that is where a visual scan happened to notice a problem. A real EDA pass checks *every* group systematically, rather than trusting that a problem will always be visible by eye:

```
def flag_outliers(group):
    q1, q3 = group.quantile(0.25), group.quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return group[(group < lower) | (group > upper)]

df_clean = df.drop_duplicates().dropna(subset=["daily_sales"])
for store, group in df_clean.groupby("store")["daily_sales"]:
    print(store, flag_outliers(group).tolist())
```

```
Downtown: no flagged values
Hilltop: 2 flagged value(s) -> [11514.6, 1398.42]
Riverside: 1 flagged value(s) -> [1640.73]
```

Common Pitfall

This systematic scan turned up something Section 5 did not mention: Riverside has a flagged value too (\$1,640.73), even though Riverside's summary statistics in Section 5 looked unremarkable. This is a genuine, unplanned finding in this reading's own dataset, not a scripted example — and it illustrates exactly why EDA should check every group the same way, rather than only investigating the group that already looks suspicious in a summary table.

6.3 From EDA Back to Interpretation

Every outlier flagged above is a *candidate*, not a confirmed conclusion. A responsible EDA write-up distinguishes clearly between what the data shows and what a human still needs to verify:

```
Interpretation: the IQR rule is a heuristic, not a certainty. The value
1398.42 at Hilltop is flagged alongside the clear 11514.60 data-entry error,
but 1398.42 could simply be a genuinely strong sales day -- exactly the kind
of judgment call that belongs to a human, per the KDD process's Interpretation
and Evaluation stage, not to the outlier rule by itself.
```

EDA also extends naturally to looking for relationships between variables, not just checking one column at a time. A **correlation matrix** summarizes how strongly each pair of numeric columns moves together, on a scale from -1 (perfectly opposite) to $+1$ (perfectly aligned):

```
wine_df[["alcohol", "color_intensity", "flavanoids", "proline"]].corr()
```

	alcohol	color_intensity	flavanoids	proline
alcohol	1.00	0.55	0.24	0.64
color_intensity	0.55	1.00	-0.17	0.32
flavanoids	0.24	-0.17	1.00	0.49
proline	0.64	0.32	0.49	1.00

Alcohol and proline correlate fairly strongly (0.64) in this real dataset, while color intensity and flavanoids correlate weakly and *negatively* (−0.17) — both genuine patterns in scikit-learn’s wine dataset, worth keeping in mind when Section 12’s capstone returns to this same data.

Practical Exercises

1. Run `df.info()` and `df.describe()` on scikit-learn’s wine dataset (loaded as a DataFrame, per Section 4). Compare what you find to the retail dataset’s profile: does the wine dataset show any missing values, duplicate rows, or an obvious outlier visible from `describe()` alone?
2. The systematic per-store outlier scan in this section found a flagged value at Riverside that was easy to miss when only looking at summary statistics. Write code that reports, for *every* numeric column in a DataFrame of your choosing (not just `daily_sales`), how many IQR-flagged outliers exist, using a loop over columns.
3. Compute the full correlation matrix (all numeric columns, not just four) for the wine dataset, and identify the single strongest positive correlation and the single strongest negative correlation *excluding* the diagonal (a column’s correlation with itself is always 1.0 and is not an interesting finding).
4. Using the retail dataset, write an EDA summary (3–5 sentences, no code required) that a store operations manager could actually read: what did you check, what did you find, and what would you recommend they do next? Avoid presenting the flagged \$11,514.60 value as a confirmed conclusion rather than a candidate needing verification.
5. Propose one additional EDA check this section did not perform on the retail dataset (for example: checking for gaps in the date range, checking whether `num_transactions` and `daily_sales` are correlated in a way that makes business sense, or checking for negative values). Implement it and report what you find.

7 Principles of Data Visualization

A chart is an argument, not just a picture[5]: it invites the viewer to draw a specific conclusion, and the same underlying numbers can be presented honestly or misleadingly depending on choices the analyst makes. Before introducing Matplotlib and Plotly, this section establishes what makes a chart trustworthy.

7.1 Choosing the Right Chart Type

The right chart depends on what question you are answering: a **line chart** shows a trend over an ordered sequence (usually time); a **bar chart** compares a quantity across discrete categories; a **histogram** shows the distribution (shape and spread) of a single numeric variable; a **scatter plot**

shows the relationship between two numeric variables; and a **box plot** compares the distribution of a numeric variable across several categories at once, summarizing median, IQR, and outliers in a single glance — the same quantities introduced in Section 5.

7.2 A Concrete Illustration of a Common Pitfall

Consider quarterly revenue for a small business: \$412K, \$418K, \$425K, \$430K. Plotted two different ways:

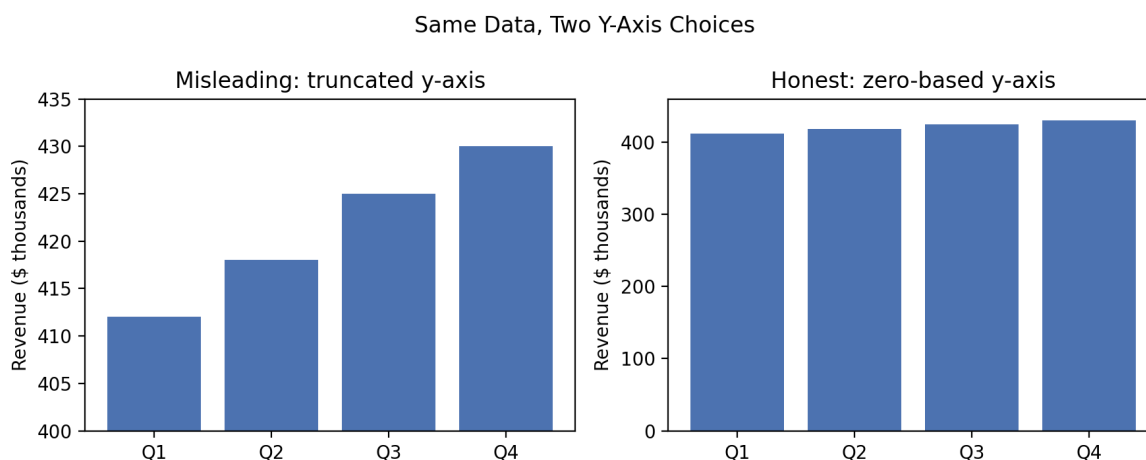


Figure 2: The same four numbers, plotted with two different y-axis choices. The left chart’s y-axis is truncated to $[400, 435]$, making quarterly growth look dramatic; the right chart’s y-axis starts at zero, showing the same growth — an honest 4.4% over the year — in its true proportion.

Common Pitfall

Both charts in Figure 2 are generated from identical, entirely accurate data. Only the y-axis range differs. The left version is not *fabricating* anything — every bar height is correct — but it is engineered to make a modest 4.4% increase look like dramatic growth, purely through axis choice. This is one of the most common ways real charts mislead: not through false data, but through a choice about presentation that a careless or motivated analyst can make without ever writing a false number.

7.3 Best Practices and Common Pitfalls

- **Start bar-chart axes at zero.** A bar’s length is visually interpreted as proportional to its value; truncating the axis (as in Figure 2’s left panel) breaks that assumption. Line charts comparing *trends* rather than absolute magnitudes are a more defensible exception, but even then, a truncated axis should be labeled clearly.
- **Match the chart type to the question**, not to whichever chart is easiest to make. A pie chart comparing many similar-sized categories, for instance, is harder to read accurately than the equivalent bar chart, because humans compare bar heights more reliably than pie-slice angles.
- **Use color with a purpose, not decoration.** Color should encode a variable (as in Figure 6’s cultivar coloring, Section 8) or highlight a specific point of interest — not vary randomly

across categories with no meaning attached.

- **Label axes and units explicitly.** A chart with an unlabeled y-axis forces the viewer to guess whether they are looking at raw counts, percentages, or something else entirely.
- **Show uncertainty and sample size where it matters.** A bar chart of “average sales” for a store with 3 data points and one with 300 look identical unless the chart (or its caption) says otherwise.

Practical Exercises

1. Using the same four quarterly revenue numbers as Figure 2 (\$412K, \$418K, \$425K, \$430K), construct a version of the misleading chart that is *even more* exaggerated (an even narrower y-axis range), and compute what percentage of the y-axis range the actual data spans in your version versus Figure 2’s original.
2. For each of the following questions, name the most appropriate chart type from this section (line, bar, histogram, scatter, or box plot) and justify your choice in one sentence: (a) “How did Downtown’s sales change day by day in June?” (b) “What is the distribution of transaction counts across all 91 store-days?” (c) “Is there a relationship between alcohol content and color intensity in the wine dataset?” (d) “How does the flavanoid/intensity ratio compare across the three wine cultivars?”
3. Find a chart from a real news article, company report, or social media post (any source) that you believe violates one of this section’s best practices. Describe the violation specifically (not just “it’s misleading”) and sketch, in words, how you would redesign it.
4. The retail dataset’s Hilltop store has one confirmed data-entry error (Section 4). Describe what would happen to a line chart of Hilltop’s daily sales if that error were left in, versus removed, and explain which version you would show to a store manager making a staffing decision, and why.
5. Pie charts are common in business presentations but are generally discouraged for anything beyond 2–3 categories. Using the three-store average-sales data from this reading, explain in 2–3 sentences why a bar chart (Section 8) would likely communicate the comparison between Downtown, Riverside, and Hilltop more accurately than a pie chart of the same three values.

8 Matplotlib for Static Visualization

Matplotlib is Python’s foundational plotting library[6]: highly customizable, widely used, and the engine underneath many higher-level libraries (Pandas’ own `.plot()` method included). This section builds the chart types introduced in Section 7 using the retail and wine datasets already familiar from earlier sections.

8.1 Basic Plotting

A line chart of one store’s daily sales over time:

```
import matplotlib.pyplot as plt

downtown = df_clean[df_clean["store"] == "Downtown"].sort_values("date")
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(downtown["date"], downtown["daily_sales"], marker="o", markersize=3)
```

```
ax.set_title("Downtown Store: Daily Sales, June 2026")
ax.set_xlabel("Date")
ax.set_ylabel("Daily Sales ($)")
fig.autofmt_xdate()
```

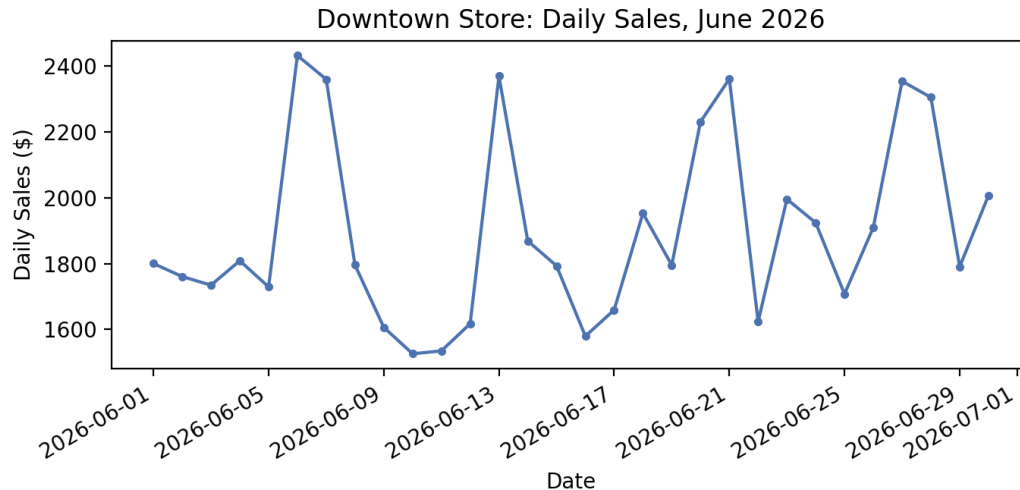


Figure 3: Downtown store’s daily sales across June 2026. The weekly pattern (a peak roughly every 7 days) is visible immediately, which is exactly the kind of pattern a table of 30 numbers would not reveal at a glance – a concrete illustration of why *Interpretation and Evaluation* (Document 1, Section 3) benefits from visualization, not just summary statistics.

8.2 Bar Charts and the Mean-Median Gap, Made Visible

```
store_avg = df_clean.groupby("store")["daily_sales"].agg(["mean", "median"])
x = np.arange(len(store_avg))
width = 0.35
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.bar(x - width/2, store_avg["mean"], width, label="Mean")
ax.bar(x + width/2, store_avg["median"], width, label="Median")
ax.set_xticks(x)
ax.set_xticklabels(store_avg.index)
ax.legend()
```

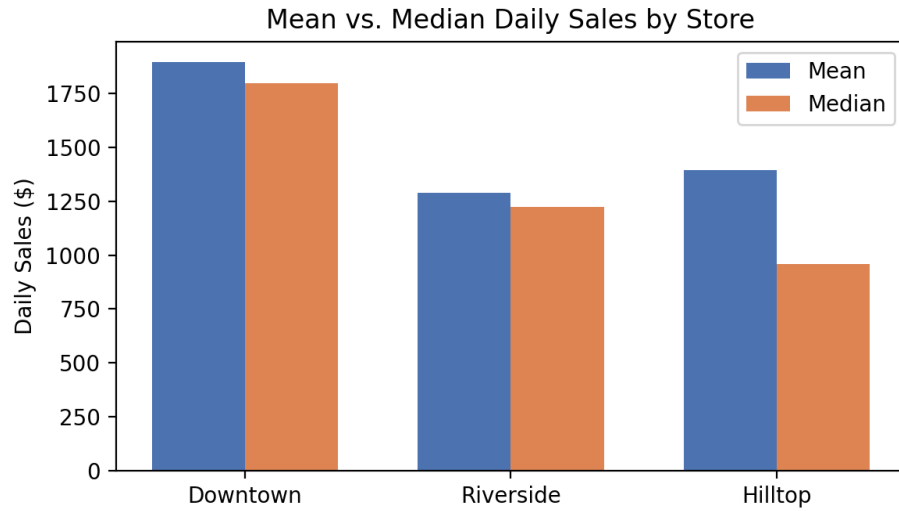


Figure 4: Mean versus median daily sales by store. Downtown and Riverside show mean and median nearly matching, while Hilltop’s bars diverge sharply – the same mean-median gap discussed numerically in Section 5, now visible at a glance without reading a single number.

8.3 Histograms: Seeing a Distribution’s Shape

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
axes[0].hist(df_clean["daily_sales"], bins=15)
axes[0].set_title("All Stores: Daily Sales (with outlier)")
axes[1].hist(df_clean[df_clean["daily_sales"] < 5000]["daily_sales"], bins=15)
axes[1].set_title("All Stores: Daily Sales (outlier excluded)")
```

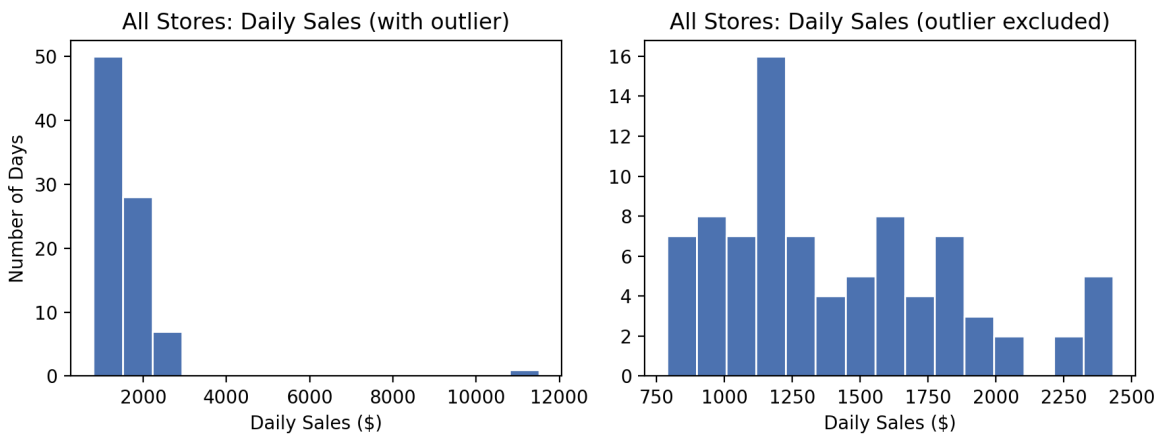


Figure 5: The full distribution of daily sales across all three stores, with and without the confirmed outlier. The left histogram compresses almost every real observation into a single bar near zero, because Matplotlib must scale its x-axis to accommodate the one \$11,514.60 value – a visual, not just numerical, demonstration of how a single outlier can distort an entire analysis.

8.4 Scatter Plots and Subplots: A Real Dataset

```
for cultivar, group in wine_df.groupby("cultivar", observed=True):
    ax.scatter(group["alcohol"], group["color_intensity"], label=cultivar, alpha=0.7)
ax.set_xlabel("Alcohol (%)")
ax.set_ylabel("Color Intensity")
ax.legend(title="Cultivar")
```

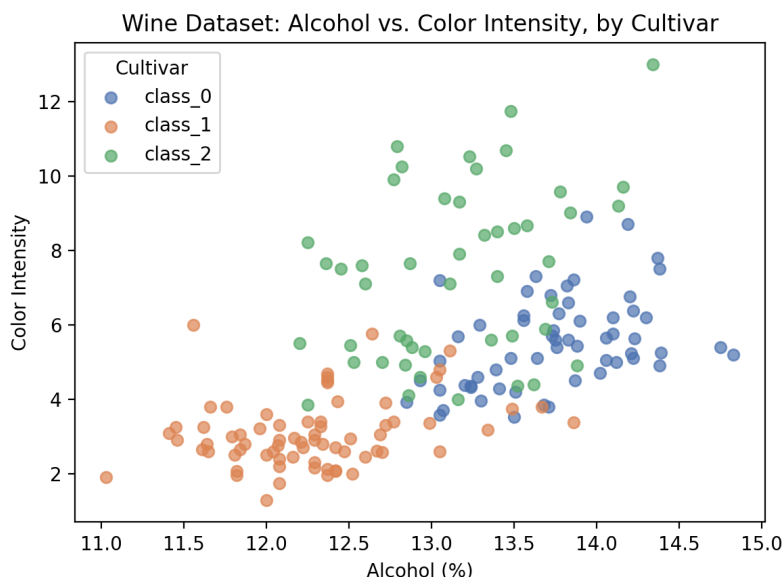


Figure 6: Alcohol content versus color intensity for scikit-learn’s wine dataset, colored by cultivar. The three cultivars separate visibly even on just these two of the dataset’s 13 chemical measurements – a real, unscripted pattern in a widely used benchmark dataset, and a preview of the kind of separation Section 12’s capstone investigates further.

Tip

Figure 6’s clean separation between cultivars did not require any modeling — just two chosen features and color. This is a recurring theme in EDA: a well-chosen visualization can reveal in seconds a pattern that would otherwise require a trained classifier to detect, and it is always worth looking before modeling.

Practical Exercises

1. Reproduce Figure 3 for the Riverside store instead of Downtown, and describe in a sentence or two how its weekly pattern compares.
2. Modify Figure 4’s bar chart to include all three summary statistics (mean, median, and standard deviation) as three grouped bars per store instead of two. You will need a third bar position and a rescaled y-axis, since standard deviation and mean/median are not necessarily the same order of magnitude for every store – discuss whether plotting all three on the same axis is actually a good idea, or whether it would violate one of Section 7’s best practices.
3. Recreate Figure 5’s histogram using 30 bins instead of 15 for the outlier-excluded panel. Does the extra granularity reveal anything the 15-bin version did not, or does it mostly

- add visual noise? Justify your answer.
4. Recreate Figure 6 using two different wine features of your choosing (not alcohol and color intensity). Do the three cultivars separate as cleanly? Report the pair of features that gives the cleanest visual separation you can find by trying at least three different feature pairs.
 5. Combine a histogram and a box plot of the same variable (Hilltop's `daily_sales`, outlier included) into a single figure with two subplots, stacked vertically, sharing the same x-axis (hint: `plt.subplots(2, 1, sharex=True)`). Explain, in your own words, what the box plot's whiskers and outlier marker show that the histogram's bars do not make as immediately obvious.

9 Plotly for Interactive Visualization

Plotly builds charts that support hovering for exact values, zooming into a region, panning, and toggling series on and off by clicking the legend[7] — interactivity Matplotlib's static images do not provide. Because this reading is a static PDF, every Plotly figure below is exported to a static image for inclusion here, but the code shown is exactly what produces a fully interactive figure when run in a Jupyter notebook or a web application.

9.1 A First Interactive Chart

Plotly Express (`plotly.express`, conventionally imported as `px`) builds common chart types in a single function call:

```
import plotly.express as px

fig = px.scatter(
    wine_df, x="alcohol", y="color_intensity", color="cultivar",
    title="Wine Dataset: Alcohol vs. Color Intensity, by Cultivar (Interactive)",
    hover_data=["flavanoids", "proline"],
)
fig.show() # opens an interactive figure in a notebook or browser
```

Wine Dataset: Alcohol vs. Color Intensity, by Cultivar (Interactive)

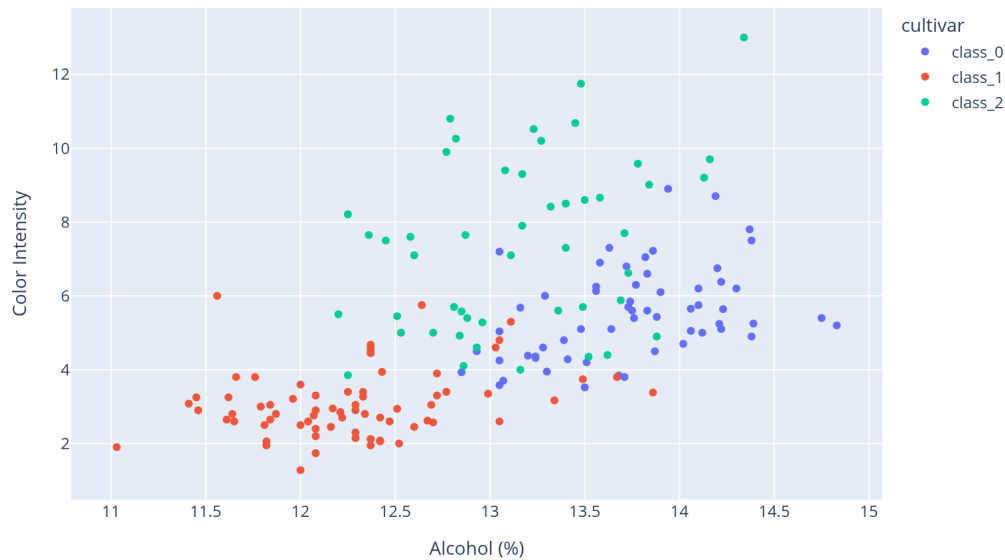


Figure 7: The same alcohol-versus-color-intensity comparison as Figure 6, built with Plotly instead of Matplotlib. In an interactive session, hovering over any point would reveal that wine’s exact flavanoid and proline values (via `hover_data`), and the legend entries could be clicked to isolate a single cultivar – neither is possible in this reading’s static image, but both are available the moment this code runs in a notebook.

9.2 Interactive Bar and Line Charts

```
store_summary = df_clean.groupby("store", as_index=False)["daily_sales"].mean()
fig2 = px.bar(store_summary, x="store", y="daily_sales", color="store",
              title="Average Daily Sales by Store (Interactive)")

fig3 = px.line(df_clean.sort_values("date"), x="date", y="daily_sales", color="store",
              title="Daily Sales Over Time, All Stores (Interactive)")
```

Average Daily Sales by Store (Interactive)

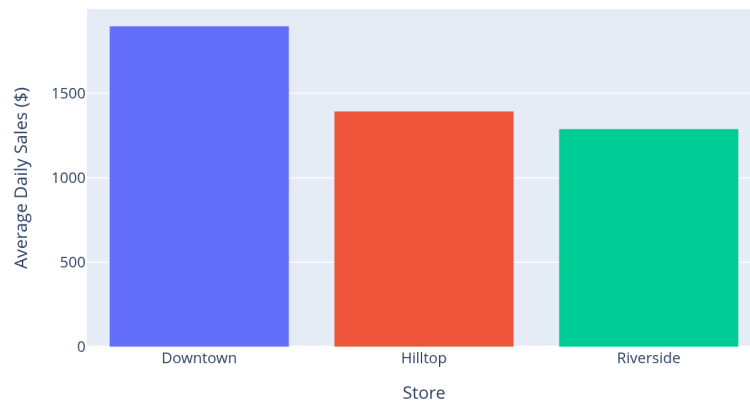


Figure 8: Average daily sales by store, interactive version. Plotly’s default styling and color palette differ from Matplotlib’s, but the underlying grammar – map a column to an axis, map a column to color – is the same idea introduced in Section 8.

Daily Sales Over Time, All Stores (Interactive)

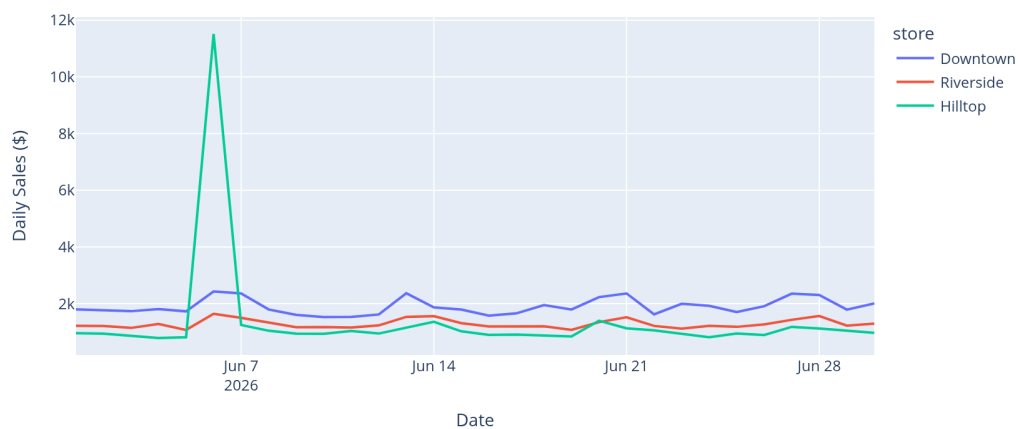


Figure 9: Daily sales over time for all three stores at once, colored by store. Hilltop’s confirmed outlier (Section 4) is now impossible to miss: it towers over every other value in the entire three-store, 30-day dataset. In an interactive session, hovering over that single point would show its exact date and value immediately – useful when a spike like this needs to be traced back to a specific transaction log.

Key Idea

Figure 9 makes a point Sections 5 and 6 already made numerically: a single outlier can dominate a chart just as thoroughly as it dominates a mean or a standard deviation. The fact that three completely different tools — a summary table (Section 4), a static histogram (Section 8), and now an interactive line chart — all surface the exact same problem is itself a useful lesson: a genuine data-quality issue tends to show up everywhere you look for it, not just in one place.

9.3 Choosing Between Matplotlib and Plotly

Neither tool is strictly better; they serve different purposes. Matplotlib produces lightweight, easily embedded static images — appropriate for a printed report, a paper, or (as in this reading) a PDF. Plotly produces richer, heavier interactive figures — appropriate for a dashboard, a notebook used for live exploration, or a web application where a viewer benefits from hovering and zooming themselves. Many data scientists use Matplotlib for finished report figures and Plotly (or a notebook’s interactive tools) during the exploratory phase, when quickly inspecting many data points by hovering is more valuable than a polished final image.

Practical Exercises

1. Recreate Figure 7 using two different wine features of your choosing, matching whichever pair you found gave the cleanest separation in Section 8’s exercises. Add a third variable to `hover_data` that was not included in this section’s version.
2. Build an interactive histogram of the retail dataset’s `daily_sales` column using `px.histogram()`, with the `color` argument set to `"store"` so all three stores appear as an overlaid or stacked histogram in one figure. Describe what this reveals that Figure 5’s two-panel Matplotlib histogram did not show as directly.
3. Using `px.box()`, build an interactive box plot of `daily_sales` grouped by `store`. Compare what this single figure communicates to the combination of Section 5’s numeric IQR calculations and Section 6’s per-store outlier scan — has Plotly’s box plot effectively done both of those steps in one call?
4. Describe a realistic scenario in this course’s suggested domains (business, healthcare, education, agriculture, finance, environmental monitoring) where you would specifically recommend Plotly’s interactivity over a static Matplotlib chart, and a different scenario where you would recommend the reverse. Justify both choices.
5. Plotly Express charts are built with far less code than the equivalent Matplotlib figure in several of this reading’s examples. Does this mean Plotly should always be preferred? Identify at least one genuine advantage Matplotlib retains despite its more verbose syntax.

10 Best Practices and Common Pitfalls

- **Vectorize before you loop.** Section 3’s 64x speedup was not a special case — any operation NumPy or Pandas provides directly will substantially outperform an equivalent hand-written Python loop over large data.
- **Never trust a mean without also checking the median.** Hilltop’s mean-median gap, first visible in Section 4’s `groupby()` output, was the single earliest warning sign of this reading’s

confirmed data-entry error — long before any chart was drawn.

- **Check every group, not just the one that looks suspicious.** Section 6’s systematic per-store scan caught a flagged Riverside value that a spot-check of Hilltop alone would have missed entirely.
- **An automated outlier flag is a candidate, not a verdict.** Section 5 and Section 6 both flagged \$1,398.42 alongside the confirmed \$11,514.60 error, and only a human, informed by domain knowledge, can tell the two apart.
- **Choose a chart’s axis range deliberately, and default to zero for bar charts.** Figure 2 showed that identical, accurate data can be made to look dramatically different through axis choice alone.
- **Match static and interactive tools to their purpose.** Matplotlib for a finished, embeddable report figure; Plotly (or an interactive notebook) for active exploration where hovering and zooming speed up the search for patterns.

11 A Practical Mental Model

Key Idea

When you receive a new dataset, resist the urge to jump straight to a model or a headline chart. Instead, run the same sequence this reading followed: load it and check its shape and types (`.info()`); compute summary statistics per group, watching specifically for mean-median gaps (Section 5); systematically scan every group for IQR-flagged outliers rather than only the one that looks suspicious (Section 6); and only then visualize, choosing the chart type the *question* calls for rather than the one that is easiest to make (Section 7). Every genuine finding in this reading — the Hilltop data-entry error, the Riverside flagged value, the wine dataset’s cultivar separation — surfaced by following exactly this sequence, in this order.

12 Capstone Exercise: An End-to-End EDA and Visualization Pipeline

This capstone applies NumPy, Pandas, descriptive statistics, EDA, and visualization together on a complete, real dataset: scikit-learn’s wine dataset[8], already familiar from Sections 4, 6, 8, and 9, now analyzed end to end.

The question: do the three cultivars differ measurably enough in their chemical profile to support a quality-control or authentication rule?

```
from sklearn.datasets import load_wine

wine_bunch = load_wine()
wine_df = pd.DataFrame(wine_bunch.data, columns=wine_bunch.feature_names)
wine_df["cultivar"] = pd.Categorical.from_codes(wine_bunch.target, wine_bunch.
    target_names)

features_of_interest = ["alcohol", "flavanoids", "color_intensity", "proline"]
selected = wine_df[features_of_interest + ["cultivar"]]
```

Preprocessing. Even a real, curated benchmark dataset should be checked, not assumed clean:

```
Missing values: {'alcohol': 0, 'flavanoids': 0, 'color_intensity': 0, 'proline': 0}
Duplicate rows: 0
```

Transformation. A derived feature — the ratio of flavanoids to color intensity — combines two measurements into one that may separate the cultivars more cleanly than either alone:

```
selected["flavanoid_intensity_ratio"] = selected["flavanoids"] / selected["
    color_intensity"]
```

Data Mining. Descriptive statistics per cultivar:

	mean	median	std
cultivar			
class_0	0.554	0.543	0.086
class_1	0.710	0.690	0.249
class_2	0.112	0.095	0.050

Visualization. Box plots make the separation immediate:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
selected.boxplot(column="alcohol", by="cultivar", ax=axes[0])
selected.boxplot(column="flavanoid_intensity_ratio", by="cultivar", ax=axes[1])
```

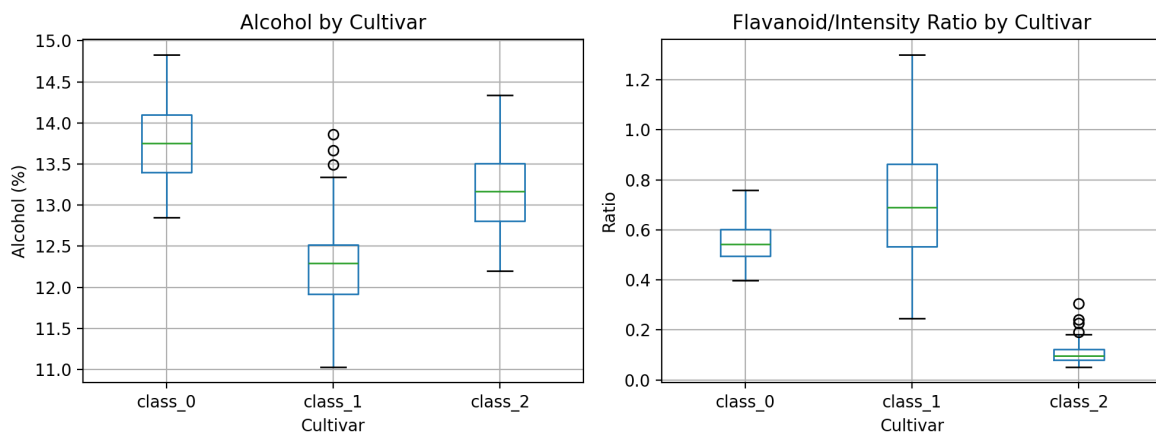


Figure 10: Alcohol content and the derived flavanoid/intensity ratio, compared across the three wine cultivars. The derived ratio (right panel) separates class_2 from the other two cultivars especially cleanly, with almost no overlap in the boxes – a stronger separation than alcohol alone (left panel) provides.

Interpretation and Evaluation.

class_1 has the highest mean flavanoid/intensity ratio (0.710), while class_2 has the lowest (0.112).

This derived feature visibly separates the cultivars in the box plot, suggesting it could support an authentication or quality-control rule -- but confirming that would require testing on new, held-out wine samples, not just describing the samples already in hand.

Common Pitfall

Notice the caveat in the last line above. This capstone found a real, useful-looking pattern in 178 wine samples — but 178 samples all measured under the same conditions is not the same as proof that the pattern would hold for a new shipment, a different vintage, or a different lab’s instruments. This is the exact caution Document 1’s bookstore capstone raised about a 10-order sample, applied here to a much larger but still finite dataset: a pattern found through EDA and visualization is a hypothesis worth testing further, not a conclusion that testing is done.

Practical Exercises

1. Reproduce this entire pipeline yourself, then extend it with one additional derived feature of your own choosing (for example, a ratio or difference between two other chemical measurements), and check whether it separates the cultivars as cleanly as `flavanoid_intensity_ratio` did.
2. The capstone’s Preprocessing step found zero missing values and zero duplicates. Discuss, in 2–3 sentences, whether this means Preprocessing was a wasted step in this particular analysis, connecting your answer to Document 1’s discussion of why KDD stages are still necessary even when they turn out to find nothing.
3. Compute the same descriptive statistics (mean, median, std) for `flavanoid_intensity_ratio` but grouped by a *different* categorical split of your own construction (for example, splitting wines into “high alcohol” versus “low alcohol” using the median alcohol value as a cutoff). Does this alternative grouping reveal anything the cultivar grouping did not?
4. Build an interactive Plotly box plot (`px.box()`) of `flavanoid_intensity_ratio` by cultivar, and compare it directly to Figure 10’s Matplotlib version. Which would you actually choose to include in a final written report to a wine producer, and which would you use while you were still actively exploring the data? Justify both choices using Section 9’s discussion.
5. This capstone’s dataset (178 real wine samples) is far larger than Document 1’s bookstore capstone (10 records) but still finite. Propose a concrete next step — beyond what this capstone did — that would strengthen confidence that the flavanoid/intensity separation is a real, generalizable pattern rather than a coincidence of this particular dataset.

13 Summary

This reading connected the KDD process introduced in Document 1 to the tools practicing data scientists actually use to carry it out. NumPy’s vectorized arrays replace the plain-Python loops of Document 1 with operations that are both faster and more concise, and underlie everything Pandas does. Pandas Series and DataFrames handle loading, selecting, filtering, cleaning, and grouping tabular data — covering Selection, Preprocessing, and Transformation at a scale plain Python could not practically reach. Descriptive statistics (central tendency and variability) and a systematic exploratory data analysis workflow turned a real, deliberately messy retail dataset’s mean-median gap into a confirmed data-entry error, demonstrating throughout that an automated statistic or outlier flag is a candidate for a human to evaluate, never a finished conclusion on its own. Visualization principles established what makes a chart trustworthy rather than merely attractive,

using a single, concrete misleading-versus-honest chart comparison; Matplotlib and Plotly then built the same core chart types — line, bar, histogram, scatter, and box plot — as static and interactive figures respectively, each best suited to a different purpose. The capstone tied every tool together on a real, bundled dataset (scikit-learn’s wine data), finding a genuine, visually striking pattern while explicitly cautioning that a pattern found through EDA is a hypothesis, not a validated finding, echoing Document 1’s closing lesson from a very different dataset and domain.

14 Discussion Questions

1. Section 3 showed a real, measured 64x speedup from vectorization. Under what circumstances (if any) might a plain Python loop still be the right choice over a vectorized NumPy operation? Consider both performance and code clarity.
2. Section 4 introduced scikit-learn’s wine dataset as a “real, bundled dataset,” in contrast to the synthetic retail dataset. What are the advantages and limitations of practicing on a small, clean, well-known benchmark dataset like this one, compared to a messy, realistic dataset like the retail sales data?
3. Section 5 and Section 6 both flagged \$1,398.42 as a candidate outlier for Hilltop, alongside the confirmed \$11,514.60 error. If you had to decide, today, whether to include or exclude \$1,398.42 from a report to store management, what additional information would you want before deciding, and why?
4. Section 6’s systematic outlier scan found a flagged value at Riverside that Section 5’s discussion did not mention. Does this change your confidence in Section 5’s conclusions? Why or why not?
5. Section 7 argued that a truncated y-axis can mislead without any single number being false. Is there a meaningful ethical difference between a truncated axis and a fabricated data point? Defend your position.
6. Section 12’s capstone found a real pattern (the flavanoid/intensity ratio separating cultivars) in a dataset of 178 samples, and cautioned against treating it as proven. How large would a dataset need to be before you would consider a similar pattern well-established, and does that number depend on the domain (business quality control vs. medical diagnosis, for instance)?

15 Key Takeaways for CIS 635

1. **NumPy and Pandas do not replace the KDD process — they implement it at scale.** Every operation in Sections 3 and 4 has a direct plain-Python equivalent from Document 1; the difference is speed and conciseness, not a new set of ideas.
2. **A mean-median gap is often the cheapest outlier detector available.** Hilltop’s disagreement between mean and median, visible from a single `groupby()` call in Section 4, was the first and cheapest signal of this reading’s confirmed data-entry error — before any dedicated outlier rule or chart was built.
3. **Systematic beats spot-checking.** Section 6’s per-group outlier scan caught a flagged Riverside value that a review of only the already-suspicious Hilltop store would have missed.

4. **An outlier flag, a correlation, and a visual pattern are all candidates for human judgment, not conclusions.** Sections 5, 6, and 12 each ended with an explicit caution to this effect, because the pattern recurs at every scale, from a 27-value column to a 178-row real dataset.
5. **A chart’s honesty depends on choices the data itself does not make.** Figure 2’s identical, accurate numbers produced two visually opposite impressions purely through axis-range choice — a reminder that visualization is an act of communication with its own ethics, not a neutral rendering of numbers.
6. **Static and interactive visualization serve different moments in an analysis.** Matplotlib’s static figures suit a finished report; Plotly’s interactivity suits active exploration — and this reading’s own genuine finding (the Hilltop and Riverside outliers) was visible in both, because it was a real pattern in the data, not an artifact of one particular tool.
7. **A real, bundled dataset is not automatically free of the concerns raised in this reading.** Even scikit-learn’s clean wine dataset still required an explicit Preprocessing check (Section 12) before its patterns could be trusted — checking is a habit, not a step reserved for data you already suspect is messy.

References

- [1] Fayyad, U. M., Piatetsky-Shapiro, G., and Smyth, P. (1996). “From Data Mining to Knowledge Discovery in Databases.” *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>
- [2] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). “Array programming with NumPy.” *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [3] McKinney, W. (2010). “Data Structures for Statistical Computing in Python.” In *Proceedings of the 9th Python in Science Conference*, 56–61. <https://conference.scipy.org/proceedings/scipy2010/mckinney.html>
- [4] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- [5] Wilkinson, L. (2005). *The Grammar of Graphics*, 2nd ed. Springer. <https://doi.org/10.1007/0-387-28695-0>
- [6] Hunter, J. D. (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [7] Plotly Technologies Inc. (2026). *Plotly Python Documentation*. <https://plotly.com/python/>
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [9] Scikit-learn developers (2026). *load_wine: Wine dataset documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html
- [10] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>